AI ENGINEERING — GRADUATE COURSE

Retrieval-Augmented Generation

From parametric limits to production-grade retrieval systems: architecture, mathematics, and engineering practice.

Dense retrieval | FAISS | BGE-M3 | Hybrid search | Evaluation

AGENDA

Course Roadmap

Part 1 — Foundations. Why LLMs need retrieval, hallucination, parametric vs non-parametric memory, RAG origins.

Part 2 — Ingestion & Retrieval. Chunking, embeddings, similarity math, FAISS, ANN search, hybrid retrieval.

Part 3 — Generation & Evaluation. Prompt construction, grounding, code walkthroughs, retrieval and generation metrics.

Part 4 — Advanced Techniques & Production. Reranking, multi-query, HyDE, GraphRAG/agentic RAG, monitoring, scaling.

Part 5 — Synthesis & Reference. Comparisons, limitations, future directions, glossary, references.

PART 1

Foundations

Why generation alone is not enough, and where RAG comes from.

MOTIVATION

The Limits of Parametric Knowledge

Frozen at training time. A model's parametric knowledge reflects its training cutoff; it cannot know about events, documents, or facts that appeared afterward.

Cannot cite sources. Answers come from weights, not from inspectable documents, so provenance and verification are impossible.

Expensive to update. Correcting or adding a single fact normally requires further training, not a simple data edit.

No access to private data. A model cannot answer questions about a company's internal documents unless that data is given to it directly.

Context windows are finite. Even very long context models cannot fit an entire knowledge base into every prompt, and long context is not used uniformly well.

DEFINITION

What Is a Hallucination?

A hallucination is a generated statement that is fluent and confident, but not supported by any real evidence.

Hallucinations are especially dangerous in knowledge-intensive tasks, where the user cannot easily tell a fabricated fact from a correct one.

Why it happens: language models are trained to produce plausible continuations of text, not to verify truth.

Why RAG helps: grounding generation in retrieved, inspectable passages gives the model real evidence to condition on, and gives the user a citation to check.

What RAG does not fix: a model can still misread or misuse a correct passage, so evaluation of the generation step remains necessary.

TWO KINDS OF KNOWLEDGE

Parametric vs Non-Parametric Memory

	Parametric memory	Non-parametric memory
What it is	Knowledge encoded in model weights	External data the model can retrieve (documents, tables, chat history)
Update cost	Requires retraining or fine-tuning	Update by editing the index, no retraining
Interpretability	Opaque, cannot be inspected	Human-readable and human-writable text
Freshness	Fixed at training time	As fresh as the underlying data source
Failure mode	Hallucination with no evidence trail	Retrieval error, but still traceable to a source

FRAMING

Context Construction as Feature Engineering

Instructions vs context. A model needs instructions on how to do a task (fixed per application) and context with the information needed for a specific query (built per query).

RAG builds context per query. Rather than stuffing the same fixed context into every prompt, RAG retrieves only what is relevant to the current question.

The classical-ML parallel. Constructing the right context for a foundation model plays the same role that feature engineering played for classical machine learning models.

Long context does not replace RAG. Context windows keep growing, but available data grows faster, and longer contexts are used less reliably by the model, with added cost and latency.

Anthropic's own guidance: if a knowledge base fits under roughly 200,000 tokens, it may be simpler to place it directly in the prompt instead of building a retrieval pipeline.

CORE DEFINITION

Defining Retrieval-Augmented Generation

RAG augments a model's generation by retrieving relevant information from an external memory source before producing an answer.

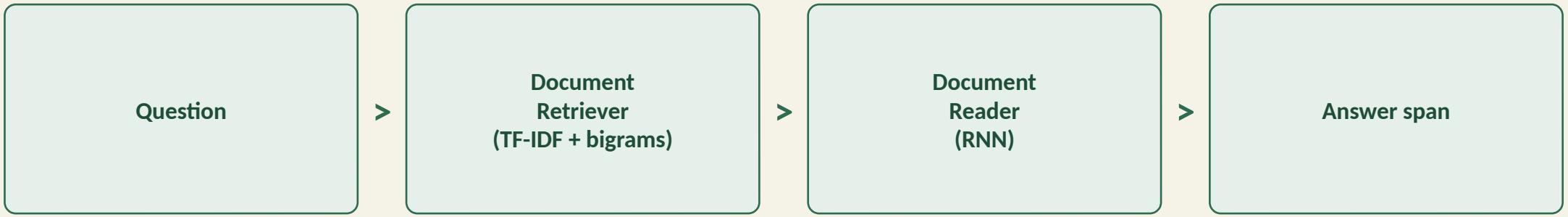
The term was coined by Lewis et al. (2020); the pattern of retrieving then reading text was already present in earlier open-domain question answering systems.

External memory can be an internal database, a user's chat history, or the open web.

Two moving parts: a retriever that finds relevant passages, and a generator that reads them to produce the answer.

Chunk terminology: a chunk of a document is itself treated as a document by the retriever, so the two terms are often used interchangeably.

Origins: the Retrieve-then-Generate Pattern



Chen et al. (2017), "Reading Wikipedia to Answer Open-Domain Questions," introduced DrQA: a Document Retriever narrows millions of Wikipedia articles down to a handful of candidates using TF-IDF and bigram hashing, and a Document Reader neural network locates the answer span inside them. This retrieve-then-read structure is the direct ancestor of modern RAG.

Origins: the RAG Paper (Lewis et al., 2020)

The contribution. A general fine-tuning recipe combining a pre-trained parametric generator (BART) with a non-parametric dense index of Wikipedia, accessed through a neural retriever (DPR).

End-to-end training. The retriever and generator are trained jointly, back-propagating through the retrieval step using Maximum Inner Product Search (MIPS) over the document index.

Result. State-of-the-art on open-domain QA benchmarks (Natural Questions, WebQuestions, CuratedTrec), and more factual, specific, and diverse generation than a parametric-only baseline on knowledge-intensive generation tasks.

Update without retraining. Swapping the document index (2016 vs 2018 Wikipedia dumps) changes what the model 'knows' without touching its weights.

TWO MARGINALIZATIONS

RAG-Sequence vs RAG-Token

$$p(y|x) \approx \Sigma z \in top\text{-}k\ p\eta(z|x) \cdot p\theta(y|x,z)$$

z is a retrieved document (or chunk), treated as a latent variable the model marginalizes over.

pη(z|x) the retriever's probability of document z given the query x (from the bi-encoder retrieval score).

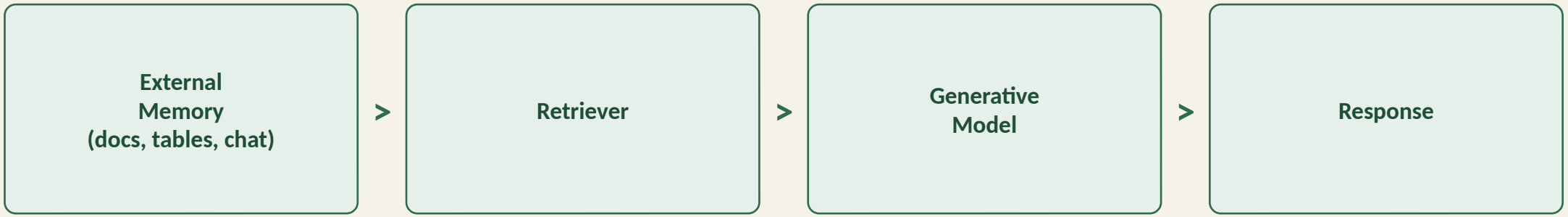
pθ(y|x,z) the generator's probability of the output y given the query and that one document.

RAG-Sequence uses the same retrieved document for the whole output sequence.

RAG-Token can draw a different document for every generated token, letting the answer combine facts from several sources.

TWO COMPONENTS

A Basic RAG Architecture



The prompt goes to both the retriever and the generator. The retriever pulls context relevant to the query from external memory; the generator conditions on the original prompt plus that retrieved context to produce the final response.

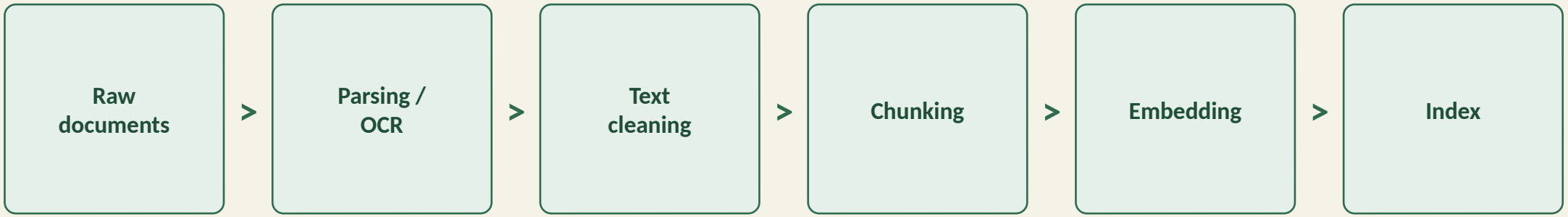
PART 2

Ingestion & Retrieval

Turning raw documents into a searchable index, and searching it well.

OVERVIEW

The Data Ingestion Pipeline



Ingestion happens once per document (or whenever documents are added or updated). Each stage can silently degrade retrieval quality if done poorly, so it deserves as much engineering attention as the model choice itself.

Document Parsing & OCR

Structured sources. PDFs, Word documents, HTML, and Markdown each need a dedicated parser to recover clean text, tables, and headings.

Scanned documents need OCR. Optical character recognition converts an image of text into machine-readable characters before any chunking or embedding is possible.

Preserve structure where it matters. Titles, section headers, and tables carry retrieval-relevant signal; flattening everything into a single blob of text discards it.

Multimodal content. Figures, charts, and images may need separate extraction pipelines (e.g., image captioning) if they should be retrievable.

Text Cleaning

Remove boilerplate. Headers, footers, navigation menus, and repeated legal text add noise without adding retrievable signal.

Normalize whitespace and encoding. Inconsistent line breaks and character encodings can silently break downstream tokenization.

Deduplicate near-identical passages. Duplicated chunks waste index space and can cause the same passage to dominate the top-k results.

Keep it lossless where possible. Aggressive cleaning can remove the very details (codes, names, numbers) a query might be searching for.

Chunking Strategies

Fixed-size

Split by a fixed unit: characters, words, sentences, or paragraphs (e.g., 512 words per chunk). Simple, fast, but can cut mid-sentence.

Recursive

Split top-down: sections, then paragraphs, then sentences, until each chunk fits the size limit. Reduces the chance of breaking related text.

Content-aware

Use splitters designed for the content: code-aware splitters for source files, question/answer pairs for FAQ documents.

Token-based

Chunk using the generator's own tokenizer as the unit, which aligns chunk size with the model's real context budget.

WHY IT MATTERS

Chunk Overlap

"I left my wife a note." → "I left my wife" + "a note."

Splitting without overlap can sever the exact information a query is looking for; neither half above preserves the original meaning.

The fix: let consecutive chunks share a small overlapping region (e.g., a 2,048-character chunk with a 20-character overlap) so boundary information survives in at least one chunk.

The cost: overlap increases the total number of chunks to embed, store, and search, so it is tuned rather than maximized.

SMALLER VS LARGER

Chunk Size Trade-offs

	Smaller chunks	Larger chunks
Diversity of retrieved information	Higher — more distinct chunks fit in the context budget	Lower — fewer, broader chunks fit
Risk of losing information	Higher — a fact split across chunks may not be retrieved as a whole	Lower — more surrounding context stays together
Indexing / storage cost	Higher — more chunks, more embeddings to store and search	Lower — fewer vectors to manage
Best fit	Precise factual lookup	Broad, narrative, or code context

There is no universal best chunk size; it is an empirical choice tuned per corpus and use case.

What Is an Embedding?

$$f(\text{text}) \rightarrow \mathbb{R}^d$$

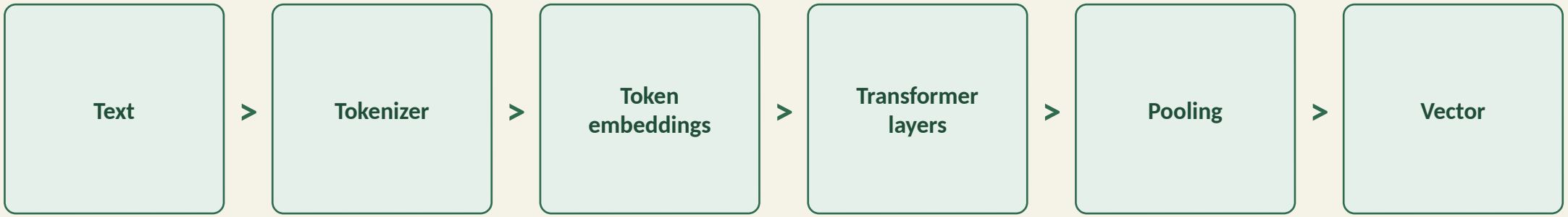
f is the embedding model: any text (a chunk, a question, a whole document) goes in, a fixed-size dense vector comes out.

d is the embedding dimension (e.g., 1024 for BGE-M3); every vector produced by f lives in the same $\mathbb{R}^d$ space.

The core property: texts with similar meaning are mapped to vectors that are close together in this space, even if they share no words.

Embeddings are not the index: f only produces vectors — a separate component (FAISS) is responsible for storing and searching them.

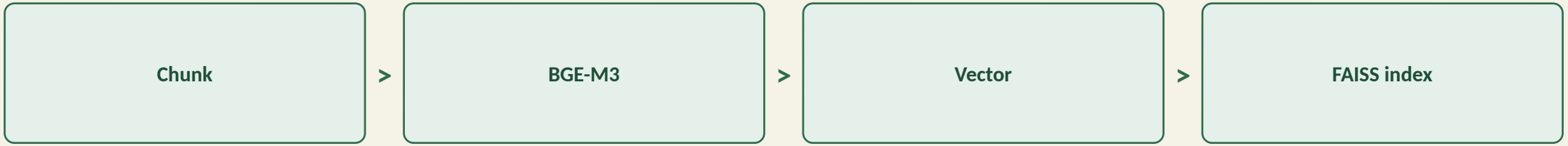
BGE-M3 Is a Deep Learning Model



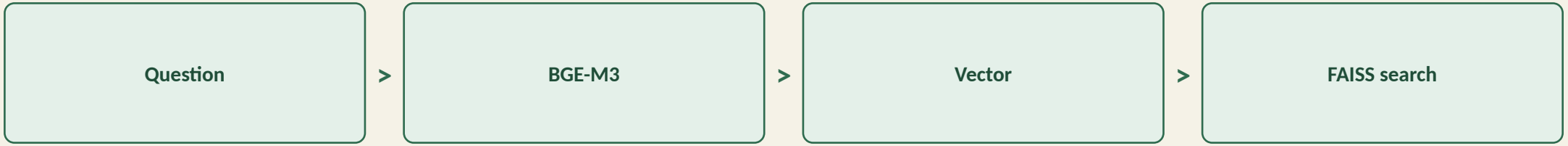
BGE-M3 is a Transformer encoder, architecturally close to BERT: it has token embeddings, positional embeddings, stacked attention and feed-forward layers, and a pooling step, all trained by gradient descent. What differs from a generative LLM is not the architecture but the training objective — pulling semantically similar texts close together in vector space, rather than predicting the next token. It is trained on a large multilingual corpus of web pages, encyclopedic text, and (query, relevant document) pairs.

Embeddings Are Used Twice

INDEXING (once per chunk)



QUERY (once per question)



SIMILARITY METRIC #1

Cosine Similarity

$$\cos(\theta) = (A \cdot B) \,/\, (\|A\| \cdot \|B\|)$$

θ the angle between the two vectors; cosine similarity measures how aligned their directions are, ignoring their length.

A · B the dot product of the two embeddings — a measure of how much they point the same way.

‖A‖, ‖B‖ the vector norms (magnitudes); dividing by them removes the effect of vector length entirely.

Range: from −1 (opposite meaning) to 1 (identical direction, i.e. very similar meaning); 0 means unrelated.

This is the default similarity for most embedding-based retrieval because it captures semantic alignment independent of how long or short a passage is.

Dot Product & Euclidean Distance

$$\text{dot: } A \cdot B = \Sigma i\ Ai\ Bi \qquad \text{L2: } d(A,B) = \sqrt{\Sigma i\ (Ai - Bi)^2}$$

Dot product is cosine similarity without the normalization step — faster to compute, but sensitive to vector magnitude, so it rewards larger-norm vectors.

Maximum Inner Product Search (MIPS) is exactly this: finding the vectors with the highest dot product against a query, the search problem underlying RAG's retriever.

Euclidean (L2) distance measures straight-line distance between two points in $\mathbb{R}^d$; smaller distance means more similar.

Practical note: if embeddings are normalized to unit length, ranking by dot product and by cosine similarity produce identical results.

Term-Based Retrieval: TF-IDF

$$Score(D,Q) = \Sigma i\ IDF(ti) \times f(ti, D) \qquad IDF(t) = log(\ N\ /\ C(t)\ )$$

f(ti, D) the term frequency: how many times query term ti appears in document D.

N, C(t) N is the total number of documents, C(t) is how many of them contain term t.

IDF(t) inverse document frequency: rare terms across the corpus (high IDF) are more informative than common ones like “the” or “for.”

Sparse vector view: each term is a one-hot vector of size = vocabulary; TF-IDF weights how much each dimension counts.

Term-Based Retrieval: BM25

$$score(D,Q) = \Sigma i\ IDF(qi) \cdot f(qi,D)(k1+1) / (f(qi,D) + k1(1-b+b \cdot |D|/avgdl))$$

k1, b tuning constants controlling term-frequency saturation and how strongly document length is penalized.

|D| / avgdl the ratio of this document's length to the average document length in the corpus.

Why it beats naive TF-IDF: BM25 normalizes term frequency by document length, so long documents no longer win purely by containing more words.

Still a strong baseline: Elasticsearch and BM25 remain widely used in production, and are often combined with dense retrieval rather than replaced by it.

Term-Based vs Embedding-Based Retrieval

	Term-based (sparse)	Embedding-based (dense)
Querying speed	Much faster	Query embedding + vector search can be slower
Out-of-the-box performance	Strong, but hard to improve further	Can outperform with fine-tuning
Weakness	Misses synonyms and paraphrases (lexical only)	Can obscure exact keywords, codes, product names
Cost	Cheap	Embedding generation, storage, and search all cost money

Vector Databases & FAISS

What a vector database does. Stores embeddings and answers nearest-neighbor queries: given a query vector, return the k closest vectors in the index.

FAISS (Facebook AI Similarity Search). A widely used open-source library for exact and approximate vector search, from a single machine up to billions of vectors.

Storing, not just searching. FAISS holds the embeddings itself; on disk this is typically saved as index.faiss, alongside a separate lookup table mapping chunk IDs to their original text.

No document text is stored in the vector. FAISS returns IDs of the nearest vectors; the calling application looks up the corresponding chunk text from its own store.

Approximate Nearest Neighbor (ANN) Search

Algorithm	Idea	Trade-off
Flat / k-NN	Compare the query to every vector exactly	Perfectly accurate, too slow at scale
IVF (inverted file)	K-means clusters vectors; search only nearby clusters	Backbone of FAISS; fast, tunable accuracy
HNSW	Multi-layer graph connecting similar vectors	High accuracy and fast queries, costly to build
LSH	Hash similar vectors into the same buckets	Fast and lightweight, trades some accuracy
Product Quantization	Compress vectors into compact sub-vector codes	Much less memory, small accuracy cost

For large datasets, exact k-NN is too slow, so production systems use an ANN index that trades a small amount of accuracy for large gains in query speed.

INDEXING CODE

FAISS in Practice

```
import faiss

# Build once: embed all chunks, then index them
embeddings = embed_model.encode(chunks)   # shape (N, d)
index = faiss.IndexFlatIP(d)              # inner product index
index.add(embeddings)
faiss.write_index(index, "index.faiss")

# Store chunk text separately (JSON / SQLite / Postgres)
save_chunk_lookup(chunk_ids, chunk_texts)

# At startup: load, do not recompute
index = faiss.read_index("index.faiss")

# At query time: embed only the question
q_vec = embed_model.encode([question])
scores, ids = index.search(q_vec, k=5)
chunks_found = [lookup[i] for i in ids[0]]
```

Why this is fast

- Document embeddings are computed once, at indexing time.
- Only the question is embedded at query time.
- FAISS never recomputes stored vectors, only compares against them.
- This is why RAG can answer in tens of milliseconds even over millions of chunks.

SPEED, PERFORMANCE, COST

Retrieval Algorithms Compared

	Term-based retrieval	Embedding-based retrieval
Querying speed	Much faster than embedding-based	Query embedding + vector search can be slow
Performance	Strong out of the box, hard to improve, can mismatch on term ambiguity	Can outperform with fine-tuning; understands natural language querie
Cost	Much cheaper	Embedding, storage, and search costs can rival model API costs

Source: adapted from Huyen, AI Engineering, Chapter 6.

Hybrid Search & Reciprocal Rank Fusion

$$Score(D) = \Sigma i\ 1\ /\ (k + ri(D))$$

Hybrid search runs term-based and embedding-based retrieval in parallel, then fuses their rankings into one.

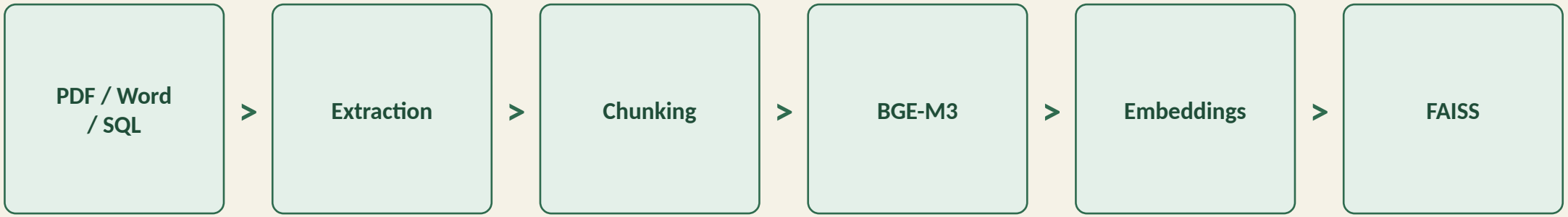
ri(D) the rank document D received from retriever i (1st, 2nd, 3rd…); n retrievers are combined by summing their contributions.

k a constant (commonly 60) that limits the influence of low-ranked documents and avoids division by zero.

Why it works: sparse retrieval catches exact keywords and codes that embeddings can blur; dense retrieval catches paraphrases that keyword search misses.

INDEXING + INFERENCE, END TO END

The Complete Retrieval Pipeline



This is the indexing half of the pipeline, run once per document. The inference half — question, embedding, FAISS search, top-k chunks, prompt construction, and generation — is covered next, in Part 3.

PART 3

Generation & Evaluation

From retrieved chunks to a grounded, citable answer.

Prompt Augmentation

Post-processing is required. Retrieved chunks are not fed to the model raw; they are joined with the user's question into a single final prompt.

Order can matter. Models tend to attend more reliably to information near the start and end of a long context, so chunk ordering is a tunable detail.

Cite while you construct. Numbering chunks as [1], [2], [3] as they are inserted makes it possible for the model to reference them directly in its answer.

Multimodal and tabular context. The same pattern extends to images (via a multimodal embedding model like CLIP) and to tables (via a text-to-SQL step instead of vector search).

PROMPT TEMPLATE

Anatomy of a RAG Prompt

```
SYSTEM
You are a scientific assistant.
Answer only using the provided context.
If the information is not present,
say that you do not know.

-------------------------
CONTEXT

[1] False positives remain a major
    challenge for ML-based IDS ...

[2] Embedded ECUs have limited
    compute resources ...

[3] Most public datasets are
    unrealistic or imbalanced ...

-------------------------
QUESTION
What are the main challenges of
ML-based IDS?

-------------------------
ANSWER
```

Design notes

- The system prompt sets the ground rule: answer only from context.
- An explicit 'say you do not know' instruction reduces hallucination on unanswerable questions.
- Numbered context blocks let the model cite [1], [2], [3] directly in prose.

COMMON MISCONCEPTION

Grounding: the LLM Never Sees Vectors

Embeddings are a search mechanism. They are never the input format for the generator.

A frequent misunderstanding is assuming the retrieved chunk vectors are passed to the LLM. They are not — only the original chunk text is.

What FAISS returns: IDs of the nearest chunks, not their vectors and not their text.

What actually reaches the LLM: the original chunk text, looked up by ID, inserted into the prompt alongside the system instructions and the question.

Why: a language model expects a sequence of tokens, and has no way to interpret a 1024-dimensional vector directly.

CODE WALKTHROUGH

Minimal RAG Pipeline — Indexing

```
from bge_m3 import BgeM3Model
import faiss

embedder = BgeM3Model()

def build_index(documents):
    chunks = []
    for doc in documents:
        text = extract_text(doc)          # parsing / OCR
        text = clean(text)                # cleaning
        chunks += chunk_text(text,
                             size=512,
                             overlap=50)  # chunking

    vectors = embedder.encode(chunks)     # (N, d)
    index = faiss.IndexFlatIP(vectors.shape[1])
    index.add(vectors)

    faiss.write_index(index, "index.faiss")
    save_chunk_store(chunks)              # JSON / SQLite
    return index
```

What happens here

- This runs once, offline, whenever the corpus changes.
- Each stage from Part 2 (parse, clean, chunk, embed, index) appears as one function call.
- The chunk store keeps the human-readable text FAISS never sees.

CODE WALKTHROUGH

Minimal RAG Pipeline — Query Time

```
def answer(question, k=5):
    q_vec = embedder.encode([question])
    scores, ids = index.search(q_vec, k)
    retrieved = [chunk_store[i] for i in ids[0]]

    context = "\n\n".join(
        f"[{i+1}] {c}" for i, c in enumerate(retrieved)
    )

    prompt = SYSTEM_PROMPT.format(
        context=context,
        question=question
    )

    return llm.generate(prompt), retrieved, scores
```

What happens here

- Only the question is embedded; document vectors were already indexed.
- Chunks are looked up by ID, never by vector.
- The function returns the answer plus its sources and similarity scores for citation.

Evaluating Retrieval: Precision & Recall

$$Precision = relevant \cap retrieved \,/\, retrieved \qquad Recall = relevant \cap retrieved \,/\, relevant$$

Context precision out of everything retrieved, what fraction is actually relevant to the query.

Context recall out of everything that is relevant in the whole corpus, what fraction was retrieved.

Computing recall is expensive: it requires knowing the relevance of every document in the database, not only the retrieved ones.

In production, many RAG evaluation frameworks report only context precision, since it only requires judging the documents actually returned.

Ranking Metrics: MRR, MAP, NDCG

$$MRR = (1/|Q|) \sum 1/rank_i \qquad NDCG = DCG / IDCG$$

MRR (Mean Reciprocal Rank) averages 1 / (rank of the first relevant result) across queries; rewards getting a relevant result near the top.

MAP (Mean Average Precision) averages precision computed at each point a relevant document appears, rewarding both precision and early ranking.

NDCG (Normalized Discounted Cumulative Gain) discounts relevance by rank position, then normalizes against the best possible ordering.

Use these whenever the position of a result, not just its presence in the top-k, affects downstream quality.

Evaluating Generation & Hallucination

End-to-end quality. The retriever is ultimately judged by whether it helps the generator produce correct, well-cited answers, not by its metrics in isolation.

Faithfulness / hallucination checks. Verify that every claim in the generated answer is actually supported by the retrieved context, typically using an LLM-as-judge or human review.

Answer relevance. Separately from faithfulness, check whether the answer actually addresses the question that was asked.

Embedding quality (MTEB). The embedding model itself can be evaluated independently across retrieval, classification, and clustering benchmarks.

A PRACTICAL DISTINCTION

Context Precision vs Context Recall

Context precision is simple to compute: compare only the retrieved documents to the query, which an AI judge can do directly.

Context recall requires labeling the relevance of every document in the database to that query, which is far more expensive.

Consequence for practitioners: most production evaluation pipelines default to context precision, and reserve recall estimation for periodic, offline audits with a curated test set.

PART 4

Advanced Techniques & Production

Beyond the basic pipeline: sharper retrieval, and running it at scale.

A SECOND, PRECISE PASS

Reranking

The pattern. A cheap, less precise retriever (BM25, or a fast dense index) fetches a wide set of candidates; a slower, more accurate model reranks them.

Why it helps. It lets you cast a wide net cheaply, then spend compute only on the smaller candidate set that actually needs precise scoring.

Common rerankers. Cross-encoder models such as a BGE reranker or Cohere Rerank score a (query, chunk) pair jointly, which is more accurate than comparing separately-computed embeddings.

Order still matters after reranking. Placing the most relevant chunks near the start or end of the final context tends to help the generator use them.

TWO WAYS TO SCORE RELEVANCE

Bi-Encoder vs Cross-Encoder

	Bi-encoder	Cross-encoder
How it scores	Encodes query and document separately, then compares vectors	Encodes query and document together, in one forward pass
Speed	Fast — documents pre-embedded once, reused for every query	Slow — every (query, document) pair needs a fresh pass
Accuracy	Good, but misses fine-grained interaction	Higher — captures word-level interaction between query and document
Typical role	First-stage retriever (BGE-M3, e5)	Second-stage reranker over a small candidate set

Multi-Query Retrieval

The idea. The LLM rewrites one query into several related phrasings, each is used to retrieve separately, and the results are merged.

Example. “AI IDS SDV” becomes “intrusion detection,” “automotive security,” “connected vehicle,” and “CAN bus IDS.”

Why it helps. A single embedding of a short, ambiguous query can miss relevant chunks phrased with different vocabulary; multiple queries widen the net.

Merging strategy. Results are typically deduplicated and combined with reciprocal rank fusion, the same technique used in hybrid search.

Query Rewriting

```
User: When was the last time John Doe
       bought something from us?
AI:   John last bought a Fruity Fedora
       hat two weeks ago.

User: How about Emily Doe?

Rewritten query (sent to the retriever):
"When was the last time Emily Doe
 bought something from us?"
```

Why rewriting is needed

- “How about Emily Doe?” is ambiguous on its own.
- Used verbatim, it would retrieve irrelevant chunks.
- An LLM rewrites it into a self-contained query before retrieval.
- If identity or facts cannot be resolved, the model should say so rather than guess.

QUERY-SIDE TRICK

HyDE — Hypothetical Document Embeddings

The problem. A short question and a long passage that answers it are not always close in embedding space, even when the passage is the right answer.

The trick. Ask the LLM to write a hypothetical answer to the question first, then embed that hypothetical passage instead of the raw question.

Why it helps. A generated passage looks structurally more like the documents in the index than a short question does, which can improve retrieval matching.

Trade-off. It adds one extra LLM call before retrieval, so it trades latency and cost for potential recall gains.

Context Compression & Parent Document Retrieval

Context compression. After retrieval, an LLM or a smaller model extracts only the sentences relevant to the query from each chunk, trimming noise before generation.

Parent document retrieval. Index small, precise child chunks for search, but return their larger parent section or document once a child chunk matches, giving the generator more surrounding context.

Why both exist. Small chunks retrieve precisely but risk losing surrounding context; these techniques let you search small while reading large.

ANTHROPIC'S TECHNIQUE

Contextual Retrieval

Each chunk is prepended with a short, LLM-generated summary situating it within its parent document before embedding. This gives standalone chunks the surrounding context they would otherwise lose, improving retrieval without changing the retrieval algorithm itself.

Emerging RAG Architectures

GraphRAG

Builds a knowledge graph over the corpus and retrieves connected entities and relations, not just similar passages — useful for multi-hop questions.

Agentic RAG

The model decides when and what to retrieve, can issue multiple retrieval calls, and can use other tools alongside retrieval.

Corrective RAG

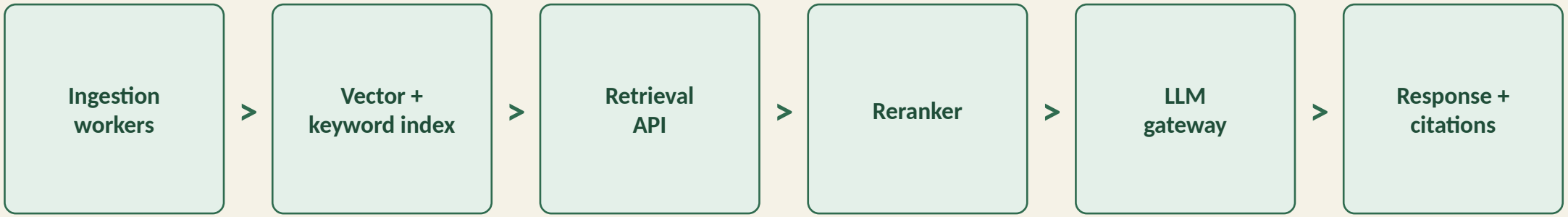
Evaluates retrieved chunks for relevance before generation, falling back to a web search or a reformulated query when retrieval quality looks poor.

Adaptive RAG

Routes each query to a different strategy (no retrieval, single-step retrieval, or multi-step retrieval) based on its estimated complexity.

END TO END

A Production RAG Architecture



In production, ingestion runs as an asynchronous batch or streaming job, decoupled from the online query path. The query path itself is usually a thin, low-latency service: retrieve, rerank, construct the prompt, call the LLM, and return the answer with citations.

Latency, Caching & Scalability

Where latency comes from. Generation of long outputs typically dominates total latency; query embedding and vector search add a comparatively small amount, but it is still user-visible.

Caching. Caching embeddings for repeated or similar queries, and caching full answers for frequently asked questions, both reduce average latency and cost.

Index rebuild cost. A detailed index (e.g., HNSW) gives high accuracy and fast queries but is slower and more memory-intensive to build; simpler indexes (e.g., LSH) build faster at the cost of query accuracy.

Scaling the vector store. Vector storage and search spending can reach a meaningful fraction of total model API spending at scale, so index choice is a real cost decision, not only a quality one.

Monitoring, Observability & Failure Modes

Track retrieval quality continuously. Sample production queries and periodically re-score context precision, not only at launch time.

Track groundedness. Monitor whether generated answers still cite and match retrieved passages as the corpus and query distribution drift over time.

Common failure modes. Stale or duplicated index entries, chunk boundaries that split key facts, retrieval collapse where the retriever always returns the same documents, and silently degraded embeddings after a model swap.

Index hot-swapping as a feature. Because non-parametric memory can be replaced without retraining, index updates should be monitored like a deployment, with the ability to roll back.

Security Considerations

Prompt injection via retrieved content. A malicious or compromised document in the index can contain instructions aimed at the model, not the user; retrieved text should be treated as untrusted data.

Data leakage across users. In multi-tenant systems, retrieval must respect access control, so one user's private documents are never retrievable by another.

Sensitive information in chunks. Ingestion pipelines need a policy for PII and secrets before they are ever embedded and indexed.

Index integrity. Write access to the vector store should be as tightly controlled as write access to a production database.

PART 5

Delivery Roadmap

Shipping a production-grade RAG system, the agile way, for organizations like Orange, Thales, or Airbus.

METHODOLOGY

Delivery Approach & Team

Framework: Scrum, 2-week sprints. A walking-skeleton MVP ships in Sprint 1; every following sprint hardens one dimension of the system (ingestion, retrieval, generation, evaluation, security, observability).

Core team. 1 Tech Lead, 1 Product Owner, 2–3 ML/data engineers, 1 backend engineer, 1 frontend engineer, 1 DevOps/MLOps engineer.

Part-time contributors. 1 security/compliance referent, 1 data governance/legal reviewer, 1 business SME per domain for evaluation and sign-off.

Ceremonies. Daily standup, sprint planning, sprint review with a live demo, retrospective; backlog refinement mid-sprint.

Definition of Ready. A ticket enters a sprint only once its acceptance criteria, data source, and access rights are confirmed — ambiguous data-access tickets are the single biggest source of delay in this kind of project.

Principle: prove the full loop end to end before optimizing any single component — a mediocre answer produced by the whole pipeline beats a perfect retriever with nothing to call it.

The Backlog, Organized in Epics

Epic	Focus
A — Platform Foundations & Security	Kubernetes, CI/CD, secrets, SSO, network isolation
B — Data Ingestion	Connectors, parsing, OCR, chunking, metadata, ACL tagging
C — Embedding & Indexing	Self-hosted embeddings, FAISS, keyword index, ACL-aware filtering
D — Retrieval Service	Hybrid search, reranking, multi-query, caching
E — Generation & Orchestration	Self-hosted LLM, prompts, guardrails, citations, memory
F — API & Frontend	Orchestration API, chat UI, source viewer, feedback capture
G — Evaluation & QA	Gold set, retrieval/generation metrics, CI quality gate
H — MLOps & Observability	Logging, dashboards, tracing, alerting, model registry, FinOps
I — Security, Compliance & Adoption	Classification, audit trail, pentest, governance, rollout

TICKETS

Backlog — Epic A: Platform Foundations & Security

ID	Ticket	Pts
A1	Set up Git repo, branching model, CI pipeline (lint / test / build)	5
A2	Provision Kubernetes clusters — dev / staging / prod	8
A3	Infrastructure as Code (Terraform + Helm charts)	8
A4	Secrets management (HashiCorp Vault)	5
A5	SSO / IAM integration (Keycloak, SAML/OIDC)	8
A6	Network isolation — no outbound calls to public AI APIs	5
A7	Private, scanned container registry & hardened base images	5

TICKETS

Backlog — Epic B: Data Ingestion

ID	Ticket	Pts
B1	Inventory & connect document sources (GED, SharePoint, file shares)	8
B2	PDF / Office parsing pipeline (PyMuPDF, unstructured)	5
B3	OCR pipeline for scanned documents	8
B4	Text cleaning & normalization module	3
B5	Configurable chunking service (recursive, overlap)	5
B6	Metadata extraction (source, date, author, classification)	8
B7	Per-document ACL / classification tagging	8
B8	Ingestion orchestration (Airflow DAGs)	8
B9	Incremental sync / change detection	5

TICKETS

Backlog — Epic C: Embedding & Indexing

ID	Ticket	Pts
C1	Deploy self-hosted embedding model (BGE-M3) endpoint	8
C2	Batch-embed the initial corpus	5
C3	Build & persist the FAISS index	5
C4	Index versioning & rollback	5
C5	Deploy keyword index (OpenSearch) alongside FAISS	8
C6	Access-control-aware filtering at index level	8

TICKETS

Backlog — Epic D: Retrieval Service

ID	Ticket	Pts
D1	Retrieval microservice (FastAPI) wrapping FAISS + OpenSearch	8
D2	Hybrid search + reciprocal rank fusion	5
D3	Cross-encoder reranker service	8
D4	Multi-query reformulation	5
D5	Retrieval result caching (Redis)	3

TICKETS

Backlog — Epic E: Generation & Orchestration

ID	Ticket	Pts
E1	Deploy self-hosted LLM via vLLM (Llama 3 / Mistral)	13
E2	Prompt template management & versioning	3
E3	Guardrails — PII redaction, injection filtering, output moderation	8
E4	Citation formatting & source linking	5
E5	Session / conversation memory	5

TICKETS

Backlog — Epic F: API & Frontend

ID	Ticket	Pts
F1	Orchestration API (FastAPI) — glue retrieval + generation	8
F2	Chat UI — Streamlit MVP, then React	8
F3	Source viewer (excerpt, page, PDF preview)	5
F4	User feedback capture (thumbs up / down)	3

TICKETS

Backlog — Epic G: Evaluation & QA

ID	Ticket	Pts
G1	Curate a gold Q&A evaluation set per business domain	8
G2	Automated retrieval evaluation (precision / recall, RAGAS)	8
G3	Automated generation evaluation (faithfulness, LLM-as-judge)	8
G4	Regression test gate in CI — blocks deploy on quality drop	5
G5	Human-in-the-loop review workflow for flagged answers	5

TICKETS

Backlog — Epic H: MLOps & Observability

ID	Ticket	Pts
H1	Centralized logging (Loki / ELK) across all stages	5
H2	Metrics & dashboards (Prometheus / Grafana)	5
H3	Distributed tracing (OpenTelemetry)	5
H4	Alerting on latency / error / quality thresholds	3
H5	Model & index registry (MLflow)	8
H6	Cost / FinOps dashboard (GPU hours, tokens)	5

TICKETS

Backlog — Epic I: Security, Compliance & Adoption

ID	Ticket	Pts
I1	Data classification policy & enforcement	8
I2	Full audit trail (query, retrieved docs, answer)	5
I3	Security review / penetration test before go-live	13
I4	GDPR & data retention policy implementation	5
I5	AI governance board review & model card	5
I6	Sovereign hosting validation (on-prem / SecNumCloud)	8
I7	Technical documentation & runbooks	5
I8	User training & onboarding materials	3
I9	Phased rollout / change management plan	5

≈ 17 WEEKS TO PRODUCTION

Sprint Roadmap

Sprint	Goal	Key tickets
0 (1 wk)	Bootstrap the platform	A1, A2, A3, A4, A7
1 (2 wk)	Walking skeleton — one query, end to end	B2, B4, B5, C1, C3, D1, E1, F1, F2
2 (2 wk)	Real ingestion at scale	B1, B3, B6, B8, B9
3 (2 wk)	Retrieval quality	C5, C6, D2, D3, D4, D5
4 (2 wk)	Generation hardening	E2, E3, E4, E5, F3, F4
5 (2 wk)	Evaluation & quality gates	G1, G2, G3, G4, G5
6 (2 wk)	Access control & compliance	B7, A5, A6, I1, I2
7 (2 wk)	Observability & MLOps	H1, H2, H3, H4, H5, H6
8 (2 wk)	Production go-live	I3, I4, I5, I6, I7, I8, I9

Sprints 4–8 run security, evaluation, and observability work in parallel with feature sprints — not sequentially at the end — to avoid last-minute go-live delays.

Tech Stack — Data & AI Layer

Layer	Technology	Why
Parsing / OCR	PyMuPDF, unstructured.io, Tesseract	Open-source and self-hostable — no document ever leaves the internal perimeter.
ETL orchestration	Apache Airflow	Mature scheduling, retries, and monitoring; already familiar to most enterprise data teams.
Embedding model	BGE-M3 (self-hosted)	Open weights (MIT), strong multilingual retrieval, dense + sparse + multi-vector in one model.
Embedding serving	Text Embeddings Inference (TEI)	Production-grade throughput for open embedding models without building a custom server.
Vector index	FAISS	Full control, proven at billion-vector scale, runs entirely on infrastructure you own.
Keyword index	OpenSearch	Enterprise-proven, native hybrid search, often already deployed internally.
Reranker	BGE-Reranker (self-hosted)	Meaningful precision gain over single-stage retrieval, no data sent externally.
LLM serving	vLLM (Llama 3 / Mistral / Qwen)	High-throughput self-hosted inference — sensitive prompts never leave the network.
Orchestration code	Thin custom Python layer	Full auditability of every call; simplifies security review versus a black-box framework.

Do You Actually Need Them?

No. This entire pipeline runs on plain Python: PyMuPDF, custom chunking, BGE-M3, FAISS, an f-string prompt, and a direct call to vLLM.

None of the three frameworks are a dependency of the architecture in this course — they are optional accelerators, not requirements. Treat each one as a build-vs-buy decision, not a default.

LlamaIndex saves time only if you have many heterogeneous sources; with 3–4 sources (GED, SharePoint, file shares), writing the parsers yourself is a day of work, not a project.

LangChain saves time only for wiring a quick prototype; on the critical path, a plain function call does the same job with nothing to audit.

LangGraph is the one exception: rebuilding a robust state machine with retries, state persistence, and human-in-the-loop for Corrective/Adaptive RAG is real engineering effort — here the framework can genuinely save weeks.

Why regulated environments often skip them anyway: a large, fast-moving dependency tree means more CVEs to track and a harder SBOM to get signed off; a security reviewer can read 200 lines of custom code far more easily than a callback-based abstraction.

Tech Stack — Platform & Enterprise Layer

Layer	Technology	Why
Hosting	On-premise, or SecNumCloud-qualified sovereign cloud	Meets ANSSI / French data-sovereignty rules for sensitive or export-controlled data.
Container orchestration	Kubernetes (often OpenShift)	Standard in large enterprise IT; strict namespace isolation and RBAC.
CI/CD	GitLab CI, self-hosted runners	Already the default in most large French enterprises; artifacts stay in-perimeter.
Secrets management	HashiCorp Vault	Centralized, auditable secret storage integrated with enterprise IAM.
Auth / SSO	Keycloak (SAML / OIDC)	Integrates with the existing corporate identity provider for per-user access control.
Observability	Prometheus, Grafana, Loki, OpenTelemetry	Open-source, self-hosted, already familiar to most enterprise SRE teams.
Model / index registry	MLflow	Tracks which model and index version produced which answer — essential for audit.
API gateway	Kong or internal enterprise gateway	Central rate limiting, authentication, and usage monitoring across business units.
Frontend	Streamlit (MVP) → React (production)	Fast to prototype, then aligned with the company's existing UX standards.

Why Everything Is Self-Hosted

Every model, every index, every document stays inside the company's own infrastructure.

For organizations handling defense, aerospace, or telecom-infrastructure data, sending prompts or documents to an external SaaS LLM API is rarely a choice — it is a line the contract and the regulator do not allow crossing.

Regulatory driver: ANSSI recommendations and SecNumCloud qualification push sensitive workloads toward sovereign or on-premise hosting.

Contractual driver: defense and export-controlled programs can prohibit any transfer of technical data outside approved, audited environments.

Practical consequence: open-weight models (BGE-M3, Llama 3, Mistral) plus self-hosted serving are the only compliant option here, not a cost-saving preference.

Definition of Done & Quality Gates

A ticket is Done only when code is reviewed and merged, CI tests pass, it is deployed to staging and smoke-tested, and — for anything touching retrieval or generation — the regression evaluation suite shows no quality drop.

A sprint is Done only when the sprint goal is demoable end to end in front of the Product Owner, not just individually tested ticket by ticket.

The system is production-ready only once the security review is signed off, the AI governance board has approved the model card, and sovereign hosting has been validated.

No ticket touching real documents is started before ACL tagging (B7) and the classification policy (I1) are in place — data governance is a blocker, not an afterthought.

Risk Register

Risk	Impact	Mitigation
Hallucination on a critical technical answer	High	Strict context-only prompting, faithfulness gate (G3), mandatory citations (E4)
Sensitive data leaked via a poisoned ingested document	High	Guardrails (E3), ACL-aware retrieval (C6), full audit trail (I2)
GPU capacity shortage for self-hosted LLM	Medium	Early capacity planning (Sprint 0), quantized models, caching (D5)
Evaluation set biased toward one business domain	Medium	Multi-domain gold set (G1), refreshed with production feedback (G5)
Security certification delays go-live	Medium	Start security & governance review in Sprint 4, not at the end
Index staleness after document updates	Low/Med	Incremental sync (B9), index versioning (C4), monitored SLAs

Perspectives: Beyond the Roadmap

Fine-tune embeddings and reranker on internal domain vocabulary (maintenance manuals, technical specs) once enough production feedback has accumulated.

Extend to GraphRAG for multi-hop technical questions that span several cross-referenced documents.

Move toward agentic RAG connected to internal systems (PLM, ERP, ticketing) so the assistant can act, not only answer.

Explore federated RAG across subsidiaries or business units, sharing retrieval capability without centralizing raw documents.

Package an offline / edge variant for field use — maintenance technicians or on-site engineers with intermittent connectivity.

Track the sovereign LLM landscape as it matures, as a lower-risk path to replace or complement the current self-hosted models.

PART 6

Synthesis & Reference

Comparisons, limitations, takeaways, and terminology.

TWO WAYS TO ADD KNOWLEDGE

RAG vs Fine-Tuning

	RAG	Fine-tuning
Update speed	Immediate — edit the index	Requires a new training run
Interpretability	High — answers cite real sources	Low — knowledge is baked into weights
Best for	Fast-changing or private factual knowledge	Teaching a model a new skill, style, or format
Cost profile	Ongoing retrieval + indexing infrastructure cost	Upfront training cost, cheaper to serve after
Can be combined	Yes — a fine-tuned model can also use RAG	Yes — same as left

Vector Database Comparison

	FAISS	ChromaDB	Qdrant
Type	Library, not a server	Lightweight embedded / server database	Full server-based vector database
Best for	Single-machine, high control, custom pipelines	Fast local prototyping	Production deployments needing filtering, sharding, and an API
Hybrid search	Manual (combine with BM25 yourself)	Basic support	Native support
Operational overhead	You manage persistence and scaling	Low for small scale	Higher, but built for scale

Embedding Model Comparison

	OpenAI embeddings	BGE-M3	E5
Access	API only, closed weights	Open weights, self-hostable	Open weights, self-hostable
Retrieval modes	Dense only	Dense, sparse, and multi-vector in one model	Dense (cross-lingual variants available)
Context length	8,191 tokens	Up to 8,192 tokens	Typically 512 tokens (base variants)
Good fit for this project	Simple to start, ongoing API cost	Self-hosted, MIT license, ready for hybrid search	Strong cross-lingual performance

Current Limitations of RAG

Retrieval failures cascade. If the retriever misses the relevant chunk, the generator has no way to recover the missing fact on its own.

Chunking is a lossy compression step. Any fixed chunking strategy will occasionally split or separate facts that belong together.

Latency and cost stack up. Embedding, vector search, optional reranking, and generation all add to the response time and the bill.

Evaluation is still hard. Faithfulness and relevance judgments often rely on LLM-as-judge methods, which are convenient but not infallible.

Future Research Directions

Tighter retriever-generator integration. Whether jointly training both components end-to-end, as in the original RAG paper, becomes standard practice again.

Better long-context and retrieval synergy. How future models might natively decide what to attend to, blurring the line between long context and explicit retrieval.

Structured and multimodal retrieval. Retrieval over knowledge graphs, tables, images, and audio in a unified framework, rather than separate ad hoc pipelines.

Standardized evaluation. More reliable, less LLM-judge-dependent benchmarks for faithfulness and citation correctness.

Practical Recommendations

Start simple. A single dense retriever, a fixed chunk size, and a basic prompt template will surface most of the real problems early.

Evaluate before optimizing. Build a small labeled test set of questions before adding reranking, hybrid search, or multi-query — you need a baseline to know if a change helped.

Log everything retrievable. Store the retrieved chunk IDs, scores, and final prompts for every production query; you cannot debug what you did not log.

Treat the index as a first-class deployment artifact. Version it, monitor it, and be ready to roll it back exactly as you would a code deployment.

SUMMARY

Key Takeaways

RAG is context construction: retrieve only what a query needs, and let the model reason over real, citable evidence.

Everything in this course composes around one separation of concerns: an embedding model turns text into vectors, a vector index compares them, and a generator turns retrieved text back into language.

Grounding beats memorization for knowledge that changes, needs citation, or must stay private.

Retrieval quality is the ceiling on generation quality — no prompt can fix a missed chunk.

The pipeline is modular and every stage, from chunking to reranking, can be measured and improved independently.

REFERENCE

Glossary (1 of 3)

Term	Definition	Why it matters	Example
Embedding	A dense vector representation of a piece of text.	It is what makes semantic (meaning-based) search possible.	“Paris” and “France’s capital” map to nearby vectors.
Vector	An ordered list of numbers representing a point in space.	Embeddings, similarity, and search are all vector operations.	A 1024-dimensional BGE-M3 output.
ANN (Approximate Nearest Neighbor)	A search that finds very likely, but not guaranteed, nearest vectors.	Makes search over millions of vectors fast enough for production.	HNSW, IVF, LSH.
FAISS	An open-source library for storing and searching vectors.	It is the most common vector search engine in RAG systems.	faiss.IndexFlatIP(d)
Retriever	The component that finds relevant chunks for a query.	Its quality caps how good the final answer can be.	A FAISS index plus an embedding model.
Generator	The language model that produces the final answer.	It reads retrieved context and writes the response.	Llama 3, Mistral, GPT.
Chunk	A manageable-sized piece of a larger document.	Documents must be split before they can be embedded and retrieved.	A 512-token paragraph from a PDF.

REFERENCE

Glossary (2 of 3)

Term	Definition	Why it matters	Example
Token	A sub-word unit a language model reads or writes.	Context length, cost, and latency are all measured in tokens.	“tokenizing” → [“token”, “izing”]
Tokenization	The process of splitting text into tokens.	It determines the model's vocabulary and its handling of rare words.	Byte Pair Encoding (BPE).
Context Window	The maximum amount of text a model can process at once.	It bounds how much retrieved context can be included per query.	128K tokens for many current models.
Hallucination	A fluent but unsupported or false generated statement.	It is the central risk RAG is designed to reduce.	Citing a paper that does not exist.
Dense Retrieval	Retrieval using dense embedding vectors.	Captures semantic similarity beyond exact keyword matches.	BGE-M3 + FAISS cosine search.
Sparse Retrieval	Retrieval using sparse, mostly-zero vectors of term weights.	Fast, cheap, and precise on exact keywords and codes.	BM25 over an inverted index.
Hybrid Search	Combining sparse and dense retrieval results.	Gets keyword precision and semantic recall together.	BM25 + FAISS, fused with RRF.

REFERENCE

Glossary (3 of 3)

Term	Definition	Why it matters	Example
Bi-Encoder	A model that encodes query and document separately.	Enables fast retrieval since documents are pre-embedded once.	BGE-M3, e5.
Cross-Encoder	A model that encodes a query and a document jointly.	More accurate scoring, used to rerank a small candidate set.	BGE Reranker, Cohere Rerank.
Reranker	A second-stage model that re-scores retrieved candidates.	Improves precision of the final top-k passed to the generator.	A cross-encoder over the top 50 results.
Grounding	Basing generation on retrieved, verifiable evidence.	It is the mechanism that reduces hallucination.	Citing [1], [2] in the final answer.
MIPS	Maximum Inner Product Search.	The formal search problem retrieval is solved as.	FAISS's IndexFlatIP.
Agentic RAG	RAG where the model actively decides when and what to retrieve.	Allows multi-step, tool-using research rather than one fixed lookup.	The model issues three separate retrieval calls before answering.

References

Chen, Fisch, Weston, Bordes (2017). “Reading Wikipedia to Answer Open-Domain Questions.” Introduced DrQA and the retrieve-then-read pattern.

Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, Küttler, Lewis, Yih, Rocktäschel, Riedel, Kiela (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” Coined the term RAG.

Huyen, C. AI Engineering, Chapter 6, “RAG and Agents.” Retrieval algorithms, vector databases, chunking, and hybrid search.

Course notes. BGE-M3 architecture, the indexing/inference separation, and the FAISS storage model used throughout Part 2.

Anthropic (2024). “Introducing Contextual Retrieval.” The chunk-context augmentation technique covered in Part 4.